Lecture 3

Embedding and Vector Representation for Language Modeling

Lecture Notes

MAI554 – Deep Learning for NLP

Prof. Anis Koubaa

College of Engineering, Alfaisal University

akoubaa@alfaisal.edu

Spring 2025

How to Use These Notes

These notes distill the key ideas from Lecture 3 into a logical flow. Each section follows a pattern: **intuition first, then math, then a concrete example**. Self-check questions appear throughout the text. Full solutions are collected in the Appendix.

Contents

1 Vectors: The Language of AI **2**
- 1.1 Why Everything Must Be a Number . . . 2
- 1.2 2D Vectors . . . 2
- 1.3 3D Vectors . . . 2
- 1.4 Representing Vectors in NumPy . . . 2
- 1.5 Feature Vectors . . . 3

2 Vector Operations **3**
- 2.1 Addition and Subtraction . . . 3
- 2.2 Scalar Multiplication . . . 3

3 Vectors in Computer Vision **4**
- 3.1 Images as Matrices of Pixel Values . . . 4
- 3.2 Example: A 2×2 Grayscale Image . . . 4
- 3.3 Color Images as 3D Tensors . . . 5

4 Vectors in NLP: Words as Vectors **5**
- 4.1 Word Embeddings . . . 5
- 4.2 Similar Words Have Similar Vectors . . . 5
- 4.3 Using spaCy to Get Embeddings . . . 5
- 4.4 Measuring Word Similarity with Dot Product . . . 6

5 The Dot Product **6**
- 5.1 Definition . . . 6
- 5.2 Worked Example . . . 6
- 5.3 Properties . . . 7
- 5.4 Geometric Meaning: Measuring Alignment . . . 7

6 Vector Normalization **7**
- 6.1 Why Normalize? . . . 7
- 6.2 L2 Norm (Euclidean Norm) . . . 8
- 6.3 L1 Norm (Manhattan Norm) . . . 8
- 6.4 When to Use Which Norm . . . 8

7 Cosine Similarity **9**
- 7.1 Definition . . . 9
- 7.2 Worked Example . . . 9
- 7.3 Why Cosine Similarity Matters . . . 10
- 7.4 Applications . . . 10

8 From Vectors to Word Embeddings **10**
8.1 The Big Picture 10
8.2 Word2Vec 11
8.3 GloVe (Global Vectors) 11
8.4 fastText 11
8.5 Comparison of Embedding Methods 12

9 Summary: The Vector Foundation for NLP **12**
9.1 Key Takeaways 12

Appendix: Solutions to Self-Check Questions **14**

Vectors: The Language of AI

Why Everything Must Be a Number

Machine learning models—neural networks, support vector machines, decision trees—are fundamentally **mathematical functions**. They can only operate on **numbers**. This means that before any AI model can process text, images, audio, or any other real-world data, we must convert that data into **numerical representations**: vectors.

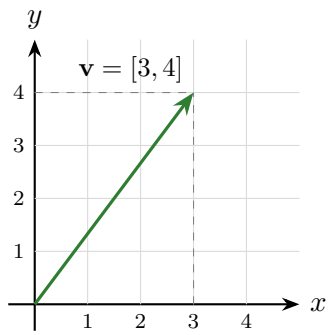
Key Insight

A **vector** is simply an ordered list of numbers. In AI, vectors are the universal language: every piece of data—a word, a sentence, an image, a sound—must be encoded as a vector before a model can process it.

2D Vectors

A **2D vector** is a list of two numbers. Geometrically, it represents a point (or a direction) in a 2D plane.

Example: $\mathbf{v} = [3, 4]$



3D Vectors

A **3D vector** extends to three dimensions: $\mathbf{v} = [x, y, z]$. In practice, vectors in AI are often hundreds or thousands of dimensions, but the same principles apply.

Representing Vectors in NumPy

Python's `NumPy` library is the standard tool for working with vectors:

```
import numpy as np

# 2D vector
v_2d = np.array([3, 4])

# 3D vector
v_3d = np.array([1, 2, 3])

# High-dimensional vector (common in NLP)
v_embed = np.array([0.2, -0.4, 0.7, 0.3, -0.1, 0.8, 0.5, -0.2])
```

Feature Vectors

In AI, a **feature vector** is an ordered list of numbers where each position represents a specific attribute of the data. For example, to represent a house for a price-prediction model:

$$\mathbf{x}_{\text{house}} = [\ \underbrace{150}_{\text{area (m}^2)},\ \underbrace{3}_{\text{bedrooms}},\ \underbrace{2}_{\text{bathrooms}},\ \underbrace{5}_{\text{age (years)}},\ \underbrace{1}_{\text{garage (yes)}}\ ]$$

Important

The **order** matters in a feature vector. Swapping two values changes the meaning entirely. All vectors in a dataset must follow the **same convention** for which position encodes which feature.

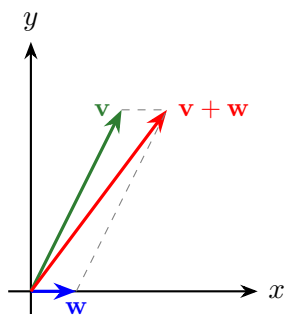
Vector Operations

Addition and Subtraction

Vector addition and subtraction are performed **element-wise**:

Worked Example:
Let $\mathbf{v} = [2, 4, 6]$ and $\mathbf{w} = [1, 0, 1]$.

$$\mathbf{v} + \mathbf{w} = [2+1,\ 4+0,\ 6+1] = [3, 4, 7]$$
$$\mathbf{v} - \mathbf{w} = [2-1,\ 4-0,\ 6-1] = [1, 4, 5]$$



Geometric intuition: Vector addition follows the **parallelogram rule**—place the tail of $\mathbf{w}$ at the head of $\mathbf{v}$, and the result is the diagonal.

Scalar Multiplication

Multiplying a vector by a scalar (a single number) scales every element:

Worked Example:
Let $c = 3$ and $\mathbf{v} = [2, 4, 6]$.

$$c \cdot \mathbf{v} = 3 \cdot [2, 4, 6] = [6, 12, 18]$$

Geometric intuition: Scalar multiplication **stretches** (if $|c| > 1$) or **shrinks** (if $|c| < 1$) the vector. A negative scalar **reverses** the direction.

```
import numpy as np

v = np.array([2, 4, 6])
w = np.array([1, 0, 1])

print(v + w)       # [3, 4, 7]
print(v - w)       # [1, 4, 5]
print(3 * v)       # [ 6, 12, 18]
```

Self-Check Q1

Given $\mathbf{a} = [1, -2, 3]$ and $\mathbf{b} = [4, 0, -1]$, compute $2\mathbf{a} + \mathbf{b}$ and $\mathbf{a} - 3\mathbf{b}$. *(Solution in Appendix)*

Vectors in Computer Vision

Images as Matrices of Pixel Values

A digital image is fundamentally a **matrix of numbers**. Each number represents the brightness (or color intensity) of a single pixel:

- **Pixel values** range from `0` (black) to `255` (white)
- **Grayscale image**: a single 2D array of shape $H \times W$
- **Color (RGB) image**: three 2D arrays stacked together, shape $H \times W \times 3$ (one channel each for Red, Green, Blue)

Example: A 2×2 Grayscale Image

Consider a tiny 2×2 image with a checkerboard pattern:

$$\text{Image} = \begin{bmatrix} 0 & 255 \\ 255 & 0 \end{bmatrix}$$

- Top-left pixel: 0 (black)
- Top-right pixel: 255 (white)
- Bottom-left pixel: 255 (white)
- Bottom-right pixel: 0 (black)

```
import numpy as np

# A 2x2 grayscale image (checkerboard)
image = np.array([[0, 255],
                  [255, 0]])

print(image.shape)    # (2, 2)
print(image.dtype)    # int64
```

Color Images as 3D Tensors

A color image has three channels. For a 4×4 RGB image:

```
# Shape: (height, width, channels)
rgb_image = np.random.randint(0, 256, size=(4, 4, 3))
print(rgb_image.shape)  # (4, 4, 3)
```

Key Insight

An image of size $224 \times 224 \times 3$ (a common input size for CNNs) is ultimately a vector of $224 \times 224 \times 3 = 150{,}528$ numbers. AI models process images by treating these numbers as input features—the same principle as feature vectors, just with many more dimensions.

Vectors in NLP: Words as Vectors

Word Embeddings

In NLP, each word is mapped to a dense vector called a **word embedding**. Unlike raw text, these vectors capture **semantic meaning**:

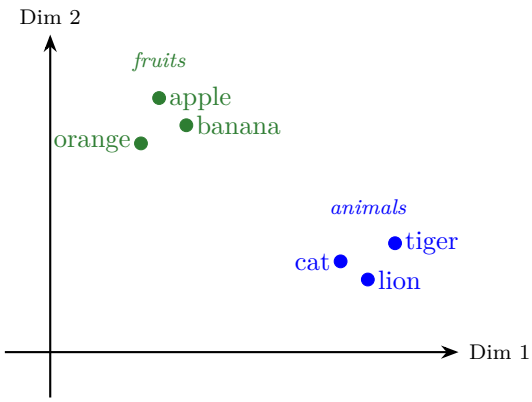
```
import numpy as np

# A 4-dimensional word embedding for "king"
king = np.array([0.2, -0.4, 0.7, 0.3])
```

Each dimension encodes some abstract linguistic feature (e.g., gender, royalty, animacy). These features are **learned automatically** from data—they are not hand-designed.

Similar Words Have Similar Vectors

The fundamental property of word embeddings is that **semantically similar words are close together** in the vector space:



Using spaCy to Get Embeddings

```
import spacy
nlp = spacy.load("en_core_web_md")

apple  = nlp("apple").vector
banana = nlp("banana").vector
```

```
lion   = nlp("lion").vector

print(apple.shape)  # (300,)  -- 300-dimensional vector
```

Measuring Word Similarity with Dot Product

We can use the **dot product** to quantify similarity between word vectors:

```
# Dot product: higher = more similar
sim_fruit = np.dot(apple, banana)   # high value (similar)
sim_diff  = np.dot(apple, lion)     # lower value (dissimilar)

print(f"apple . banana = {sim_fruit:.2f}")
print(f"apple . lion   = {sim_diff:.2f}")
```

Intuition

Words that appear in similar contexts ("I ate an *apple*" vs. "I ate a *banana*") end up with similar vectors. Words from different semantic domains ("apple" vs. "lion") end up far apart. This is the distributional hypothesis: *"You shall know a word by the company it keeps"* (Firth, 1957).

Self-Check Q2

Why do word embeddings typically have 100–300 dimensions instead of, say, 3 or 10,000? What are the trade-offs? *(Solution in Appendix)*

The Dot Product

Definition

The **dot product** (also called the **inner product**) of two vectors $\mathbf{x}$ and $\mathbf{y}$ of length n is:

$$\boxed{\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i \cdot y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n} \tag{1}$$

The dot product takes two vectors and returns a **single number** (a scalar).

Worked Example

Let $\mathbf{x} = [2, 3, 4]$ and $\mathbf{y} = [1, 0, -1]$.

$$\mathbf{x} \cdot \mathbf{y} = (2)(1) + (3)(0) + (4)(-1) = 2 + 0 - 4 = -2$$

```
import numpy as np

x = np.array([2, 3, 4])
y = np.array([1, 0, -1])

print(np.dot(x, y))  # -2
```

Properties

- **Commutative:** $\mathbf{x} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{x}$
- **Distributive:** $\mathbf{x} \cdot (\mathbf{y} + \mathbf{z}) = \mathbf{x} \cdot \mathbf{y} + \mathbf{x} \cdot \mathbf{z}$
- **Scalar factor:** $(c\,\mathbf{x}) \cdot \mathbf{y} = c\,(\mathbf{x} \cdot \mathbf{y})$

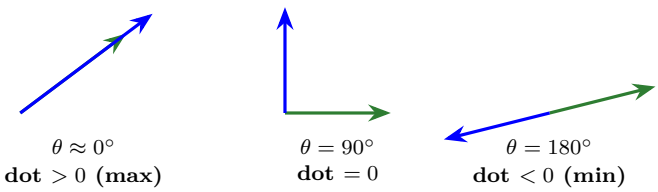
Geometric Meaning: Measuring Alignment

The dot product has a powerful geometric interpretation:

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \cdot \|\mathbf{y}\| \cdot \cos\theta \tag{2}$$

where θ is the angle between the two vectors. This tells us:

Case	Angle	Dot Product	Meaning
Same direction	$\theta = 0°$	Maximum (positive)	Perfectly aligned
Perpendicular	$\theta = 90°$	0	No relationship
Opposite direction	$\theta = 180°$	Maximum (negative)	Perfectly opposed
Some angle	$0° < \theta < 90°$	Positive	Partially aligned



Why This Matters for NLP

In word embeddings, the dot product measures how "aligned" two word meanings are. A high positive dot product means the words are semantically similar. A dot product near zero means they are unrelated. A negative dot product means they are semantically opposed.

Self-Check Q3

Compute the dot product of $\mathbf{a} = [1, 2, 3]$ and $\mathbf{b} = [4, -5, 6]$. Are these vectors more aligned, perpendicular, or opposed? *(Solution in Appendix)*

Vector Normalization

Why Normalize?

Consider two word vectors:

- $\mathbf{a} = [100, 200]$ (a word that appears 1000 times in the corpus)
- $\mathbf{b} = [1, 2]$ (a rare word)

Their dot product $\mathbf{a} \cdot \mathbf{b} = 500$ is large simply because $\mathbf{a}$ has large values, not because they are particularly similar. **Normalization** removes the effect of magnitude so we can compare **directions only**.

L2 Norm (Euclidean Norm)

The **L2 norm** measures the "length" of a vector:

$$\boxed{\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}} \tag{3}$$

Worked Example:

Let $\mathbf{v} = [3, 4]$.

$$\|\mathbf{v}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Normalized vector (unit vector):

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{[3, 4]}{5} = [0.6,\ 0.8]$$

The normalized vector has length 1: $\sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = 1$.

```
import numpy as np

v = np.array([3, 4])
norm = np.linalg.norm(v)            # 5.0
v_normalized = v / norm              # [0.6, 0.8]
print(np.linalg.norm(v_normalized))  # 1.0
```

L1 Norm (Manhattan Norm)

The **L1 norm** sums the absolute values:

$$\boxed{\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i| = |x_1| + |x_2| + \cdots + |x_n|} \tag{4}$$

Example: $\mathbf{v} = [3, -4]$, then $\|\mathbf{v}\|_1 = |3| + |-4| = 7$.

When to Use Which Norm

Norm	Geometric Meaning	Common Use
L2 (Euclidean)	Straight-line distance	Cosine similarity, embeddings
L1 (Manhattan)	City-block distance	Sparse models, regularization

Key Point

After L2 normalization, every vector has length 1. This means the dot product of two normalized vectors equals their **cosine similarity**—it measures only the **angle** between vectors, ignoring magnitude.

Self-Check Q4

Compute the L2 norm and the L1 norm of $\mathbf{u} = [1, -2, 2]$. Then normalize $\mathbf{u}$ using the L2 norm. *(Solution in Appendix)*

Cosine Similarity

Definition

Cosine similarity measures the cosine of the angle between two vectors, giving a value between -1 and $+1$:

$$\boxed{\text{cos_sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \cdot \|\mathbf{y}\|_2} = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \cdot \sqrt{\sum_{i=1}^{n} y_i^2}}} \tag{5}$$

Value	Meaning	Example
$+1$	Identical direction	"happy" and "joyful"
0	Orthogonal (unrelated)	"happy" and "table"
-1	Opposite direction	"happy" and "sad"

Worked Example

Let $\mathbf{a} = [3, 4]$ and $\mathbf{b} = [4, 3]$.

Step 1: Dot product.

$$\mathbf{a} \cdot \mathbf{b} = (3)(4) + (4)(3) = 12 + 12 = 24$$

Step 2: Norms.

$$\|\mathbf{a}\|_2 = \sqrt{9 + 16} = 5, \qquad \|\mathbf{b}\|_2 = \sqrt{16 + 9} = 5$$

Step 3: Cosine similarity.

$$\text{cos_sim}(\mathbf{a}, \mathbf{b}) = \frac{24}{5 \times 5} = \frac{24}{25} = 0.96$$

This is very close to 1, meaning $\mathbf{a}$ and $\mathbf{b}$ point in nearly the same direction.

```
import numpy as np

a = np.array([3, 4])
b = np.array([4, 3])

cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(f"Cosine similarity: {cos_sim:.4f}")  # 0.9600
```

Why Cosine Similarity Matters

Direction vs. Magnitude

Cosine similarity measures how similar two vectors are **in direction**, regardless of their magnitude. This is crucial because:

- A document about "cats" that is 10 pages long should be as similar to another "cats" document as one that is 2 pages long—they point in the same direction even though the longer document has larger vector magnitude.
- Word frequency differences should not affect semantic similarity comparisons.

Applications

- **Document similarity:** Represent documents as vectors (e.g., TF-IDF), compare with cosine similarity
- **Word similarity:** Compare word embeddings to find synonyms
- **Search engines:** Rank documents by cosine similarity to the query vector
- **Recommendation systems:** Find users/items with similar preference vectors

Self-Check Q5

Compute the cosine similarity between $\mathbf{p} = [1, 0, -1]$ and $\mathbf{q} = [-1, 0, 1]$. What does the result tell you about the relationship between these vectors? *(Solution in Appendix)*

From Vectors to Word Embeddings

The Big Picture

Now that we understand vectors and similarity measures, we can address the central question: **How do we convert words into meaningful vectors?**

There are two major approaches:

Approach 1: One-Hot Encoding (Naive)

Each word is represented as a vector with a single 1 and all other entries 0:

$$\text{“cat”} = [1, 0, 0, 0, \ldots, 0], \quad \text{“dog”} = [0, 1, 0, 0, \ldots, 0], \quad \text{“fish”} = [0, 0, 1, 0, \ldots, 0]$$

Problems with One-Hot Encoding

- **Huge dimensionality:** Vocabulary of 50,000 words → each vector has 50,000 dimensions
- **Sparse:** Almost all entries are 0 (wasteful)
- **No semantics:** cos_sim("cat", "dog") = 0 — same as cos_sim("cat", "refrigerator") = 0. The model cannot tell that cats and dogs are more related than cats and refrigerators.

Approach 2: Dense Embeddings (Modern)

Each word is mapped to a **low-dimensional, dense vector** (typically 100–300 dimensions) that captures semantic meaning. These vectors are **learned from data**.

$$\text{“cat”} = [0.2, -0.4, 0.7, \ldots], \quad \text{“dog”} = [0.25, -0.38, 0.65, \ldots]$$

Now $\text{cos_sim}(\text{“cat”}, \text{“dog”}) \approx 0.85$ — the model understands they are related.

Word2Vec

Word2Vec (Mikolov et al., 2013) learns word embeddings by training a shallow neural network on a simple prediction task. It has two variants:

CBOW (Continuous Bag of Words)

Predict the **center word** from surrounding context words:

$$\text{Context: “The _____ sat on the mat”} \rightarrow \text{Predict: “cat”}$$

Skip-gram

Predict the **context words** from the center word:

$$\text{Input: “cat”} \rightarrow \text{Predict: “The”, “sat”, “on”, “the”, “mat”}$$

The Famous Analogy

Word2Vec captures remarkable semantic relationships through vector arithmetic:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

This works because the model learns that the “royalty” and “gender” dimensions are independent, so subtracting “man” removes the male component, and adding “woman” adds the female component.

GloVe (Global Vectors)

GloVe (Pennington et al., 2014) takes a different approach: instead of predicting context words, it factorizes the **global word co-occurrence matrix**. If “ice” and “cold” frequently appear together across the entire corpus, GloVe ensures their vectors are close.

- **Advantage:** Captures global corpus statistics, not just local windows
- **Result:** Often produces high-quality embeddings for common words

fastText

fastText (Bojanowski et al., 2017) extends Word2Vec by representing each word as a **bag of character n-grams** (subwords). For example, “where” with $n = 3$:

$$\text{“where”} \rightarrow \{\text{“<wh”}, \text{“whe”}, \text{“her”}, \text{“ere”}, \text{“re>”}\}$$

- **Advantage:** Can generate vectors for **out-of-vocabulary** (OOV) words by composing subword vectors
- **Example:** Even if “unfriendliness” was never seen during training, fastText can construct its vector from subwords like “un”, “friend”, “li”, “ness”

Comparison of Embedding Methods

	Word2Vec	**GloVe**	**fastText**
Training	Predict context (local windows)	Co-occurrence matrix (global)	Predict context + subwords
OOV words	Cannot handle	Cannot handle	Handles via subwords
Morphology	Ignores	Ignores	Captures ("run", "running")
Speed	Fast	Medium	Fast
Best for	General use	Large corpora	Morphologically rich languages

Limitation of All Three Methods

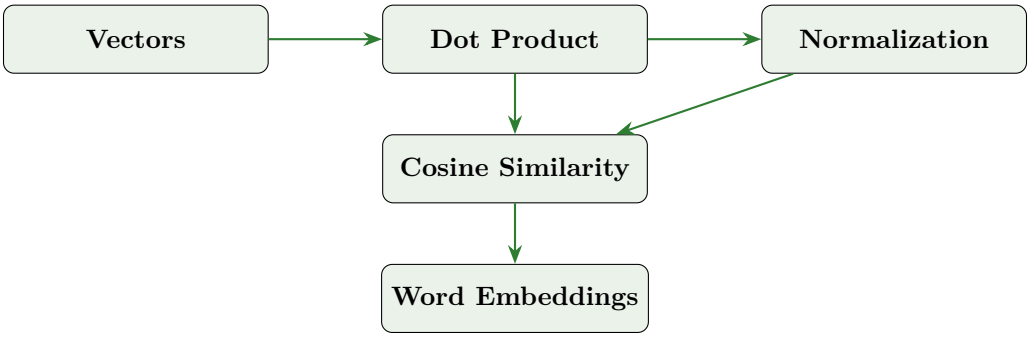
Word2Vec, GloVe, and fastText all produce **static embeddings**: each word gets one fixed vector regardless of context. The word "bank" has the same vector whether it means a financial institution or a riverbank. Solving this limitation requires **contextual embeddings** (ELMo, BERT, GPT), which are the subject of Lecture 4.

Self-Check Q6

Explain why the analogy $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$ works in Word2Vec. What geometric property of the embedding space does this exploit? *(Solution in Appendix)*

Summary: The Vector Foundation for NLP

This lecture established the **mathematical foundation** that underpins all of modern NLP:



Key Takeaways

1. **Vectors** are the universal representation for all data in AI—text, images, audio.
2. **Vector operations** (addition, subtraction, scalar multiplication) enable arithmetic on meanings.
3. The **dot product** measures alignment between vectors; it is the core operation of similarity computation.

4. **Normalization** removes magnitude effects, enabling fair direction-based comparison.
5. **Cosine similarity** combines dot product and normalization to give a principled similarity measure in $[-1, 1]$.
6. **Word embeddings** (Word2Vec, GloVe, fastText) use these mathematical tools to map words into meaningful vector spaces where similar words are close together.
7. All these methods produce **static** embeddings. **Contextual** embeddings (Lecture 4) will address the limitation that each word gets a single fixed vector.

Self-Check Q7

A search engine represents both documents and queries as vectors. Explain step by step how it uses the concepts from this lecture to rank documents by relevance to a query. *(Solution in Appendix)*

Appendix: Solutions to Self-Check Questions

Q1: Compute $2\mathbf{a} + \mathbf{b}$ and $\mathbf{a} - 3\mathbf{b}$

Given $\mathbf{a} = [1, -2, 3]$ and $\mathbf{b} = [4, 0, -1]$:

$2\mathbf{a} + \mathbf{b}$:

$$2\mathbf{a} = [2, -4, 6], \qquad 2\mathbf{a} + \mathbf{b} = [2 + 4,\ -4 + 0,\ 6 + (-1)] = [6, -4, 5]$$

$\mathbf{a} - 3\mathbf{b}$:

$$3\mathbf{b} = [12, 0, -3], \qquad \mathbf{a} - 3\mathbf{b} = [1 - 12,\ -2 - 0,\ 3 - (-3)] = [-11, -2, 6]$$

Q2: Why 100–300 dimensions for word embeddings?

This is a balance between **expressiveness** and **efficiency**:

- **Too few dimensions (e.g., 3–10):** Not enough capacity to capture the rich semantic relationships among tens of thousands of words. Different meanings would "collide" in the same region of space.
- **Too many dimensions (e.g., 10,000):** The model has more parameters to learn, requiring much more training data and computation. It also risks overfitting—memorizing the training data rather than learning general patterns. Additionally, high-dimensional spaces suffer from the **curse of dimensionality**: distances between points become less meaningful.
- **100–300 dimensions:** Empirically found to be the sweet spot—enough capacity to represent nuanced semantic relationships while remaining computationally tractable.

Q3: Dot product of $\mathbf{a} = [1, 2, 3]$ and $\mathbf{b} = [4, -5, 6]$

$$\mathbf{a} \cdot \mathbf{b} = (1)(4) + (2)(-5) + (3)(6) = 4 - 10 + 18 = 12$$

Since the dot product is **positive**, the vectors are **partially aligned** (the angle between them is less than 90°). They are not perfectly aligned (which would require them to point in exactly the same direction) but they are more similar than different.

Q4: L2 norm, L1 norm, and normalization of $\mathbf{u} = [1, -2, 2]$

L2 norm:

$$\|\mathbf{u}\|_2 = \sqrt{1^2 + (-2)^2 + 2^2} = \sqrt{1 + 4 + 4} = \sqrt{9} = 3$$

L1 norm:

$$\|\mathbf{u}\|_1 = |1| + |-2| + |2| = 1 + 2 + 2 = 5$$

L2-normalized vector:

$$\hat{\mathbf{u}} = \frac{\mathbf{u}}{\|\mathbf{u}\|_2} = \frac{[1, -2, 2]}{3} = \left[\frac{1}{3}, -\frac{2}{3}, \frac{2}{3}\right] \approx [0.333, -0.667, 0.667]$$

Verification: $\sqrt{(1/3)^2 + (-2/3)^2 + (2/3)^2} = \sqrt{1/9 + 4/9 + 4/9} = \sqrt{9/9} = 1.$ ✓

Q5: Cosine similarity of $\mathbf{p} = [1, 0, -1]$ and $\mathbf{q} = [-1, 0, 1]$

Dot product:

$$\mathbf{p} \cdot \mathbf{q} = (1)(-1) + (0)(0) + (-1)(1) = -1 + 0 - 1 = -2$$

Norms:

$$\|\mathbf{p}\|_2 = \sqrt{1 + 0 + 1} = \sqrt{2}, \qquad \|\mathbf{q}\|_2 = \sqrt{1 + 0 + 1} = \sqrt{2}$$

Cosine similarity:

$$\text{cos_sim}(\mathbf{p}, \mathbf{q}) = \frac{-2}{\sqrt{2} \cdot \sqrt{2}} = \frac{-2}{2} = -1$$

A cosine similarity of -1 means the vectors point in **exactly opposite directions**. $\mathbf{q} = -\mathbf{p}$, so they are perfectly anti-correlated. In an NLP context, this would mean the two items have maximally opposite meanings (though in practice, cosine similarities of exactly -1 are rare for word embeddings).

Q6: Why does king − man + woman ≈ queen?

Word2Vec learns an embedding space where **semantic relationships correspond to consistent vector offsets**. The key geometric property is that analogous relationships form **parallelograms** in the vector space:

- The vector from "man" to "king" captures the concept of **royalty**: $\vec{\text{king}} - \vec{\text{man}} \approx \vec{\text{royalty}}$
- The vector from "woman" to "queen" captures the **same concept**: $\vec{\text{queen}} - \vec{\text{woman}} \approx \vec{\text{royalty}}$
- Therefore: $\vec{\text{king}} - \vec{\text{man}} \approx \vec{\text{queen}} - \vec{\text{woman}}$
- Rearranging: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

This works because the embedding space organizes words such that semantic dimensions (gender, royalty, tense, etc.) are **approximately linear and additive**. This is an emergent property of training on large corpora—the model discovers these axes of meaning automatically.

Q7: How a search engine uses these concepts

A vector-based search engine works as follows:

1. **Vectorize documents:** Each document in the collection is converted into a vector. This can be done using TF-IDF (term frequency–inverse document frequency) or by averaging word embeddings for all words in the document.
2. **Vectorize the query:** The user's search query is converted into a vector using the same method.
3. **Normalize:** Both document vectors and the query vector are L2-normalized so that document length does not bias the results (a 100-page document about cats should not outrank a 2-page document about cats just because it is longer).
4. **Compute cosine similarity:** For each document, compute $\text{cos_sim}(\vec{q}, \vec{d})$. Since both vectors are normalized, this is simply the dot product $\vec{q} \cdot \vec{d}$.
5. **Rank:** Sort documents by cosine similarity in descending order. The document with the highest cosine similarity is the most relevant to the query.
6. **Return top-k:** Present the top-k ranked documents to the user.

This approach captures **semantic relevance** (not just keyword matching) because the embedding vectors encode meaning, and cosine similarity measures how closely that meaning aligns between query and document.